Translating Cost-Accounted Rho Calculus Back to the Pure Rho Calculus

Toward Rearchitecting Phlogiston Accounting in Rholang

L. Gregory Meredith*

April 2026

Abstract

The rho calculus, the formal core of the Rholang smart-contract language, is a reflective higher-order process calculus in which channels are the quoted codes of processes and processes may be recovered from channels by dequotation. Rholang's current cost-accounting mechanism — the *phlogiston* (phlo) system — is specified as an extension of the calculus with digital signatures and token-carrying rewrite rules. In this note we show that the entire cost-accounting layer can be *translated back* into the pure rho calculus via a compositional encoding. Signatures become channels, tokens become messages on those channels, and signed processes must consume "fuel" before they can communicate. Five mediator infrastructure processes (splitters and joiners) handle the bookkeeping between combined and decomposed token representations.

Because the rho calculus is the formal core of Rholang, this translation doubles as a blueprint for rearchitecting phlogiston accounting in Rholang 1.2 and beyond: cost control is expressible *within* the language, eliminating the need for a privileged runtime extension and opening the door to user-definable fee structures, composable metering contracts, and formal verification of the accounting layer itself.

Contents

1 Introduction **2**

2 The Pure Rho Calculus **2**
- 2.1 Syntax . . . 3
- 2.2 Free Names . . . 3
- 2.3 Structural Equivalence . . . 3
- 2.4 Rewrite Rules (Operational Semantics) . . . 3

3 The Cost-Accounted Extension **4**
- 3.1 Extended Syntax . . . 4
- 3.2 Cost-Accounted Rewrite Rules . . . 4

4 The Translation **5**
- 4.1 Signatures to Names . . . 5
- 4.2 Tokens to Processes . . . 5
- 4.3 Signed Processes . . . 5
  - 4.3.1 Atomic Signatures . . . 5
  - 4.3.2 Compound Signatures . . . 5
- 4.4 System Translation . . . 6
- 4.5 Infrastructure Processes . . . 6

*F1R3FLY.io; F1R3FLY Industries. lgreg.meredith@f1r3fly.io

5 Verification **6**
5.1 Rule 1: Single Signature, Whole Redex . . . 6
5.2 Rule 2: Compound Signature, Whole Redex, Split Tokens . . . 7
5.3 Rule 3: Compound Signature, Whole Redex, Combined Token . . . 7
5.4 Rule 4: Split Processes, Combined Token . . . 7
5.5 Rule 5: Split Processes, Split Tokens . . . 7

6 Implications for Rholang Architecture **8**
6.1 Cost Control as a Rholang Program . . . 8
6.2 Composable Metering . . . 8
6.3 Formal Verification . . . 8
6.4 Implementation Path . . . 8

7 Conclusion **9**

1 Introduction

The rho calculus [1] is a variant of Milner's π-calculus [2] distinguished by its support for *reflection*: processes can be turned into channels (via the quoting operator $@\cdot$) and channels can be turned back into processes (via the dequoting operator $*\cdot$). This reflective structure makes the rho calculus especially well-suited as the formal core of a smart-contract language, and indeed it is the theoretical foundation of **Rholang** [3], the native language of the RChain / F1R3FLY platform.

In production, every Rholang deploy is gated by a cost-accounting mechanism: the *phlogiston* (phlo) system. A deploy is signed by one or more digital signatures, each associated with a *token balance*; execution consumes phlo proportionally to the resources used. The current implementation treats cost accounting as a runtime concern, external to the calculus itself.

This paper demonstrates that the cost-accounting extension can be faithfully *encoded* in the pure rho calculus. Because the rho calculus is the core of Rholang, the encoding immediately suggests a rearchitecture of phlogiston accounting:

(i) **Cost control within the language.** Rather than a privileged runtime extension, phlo accounting becomes a Rholang program, subject to the same formal semantics and verification tools as any other contract.

(ii) **Composable metering.** User-definable fee structures and composable metering contracts become possible, because the accounting layer is written in the same language as the contracts it meters.

(iii) **Formal verification.** The translation is compositional and can be verified using the standard bisimulation and full-abstraction proof techniques of the rho calculus, including the ongoing Lean 4 formalization effort.

The remainder of the paper is organized as follows. Section 2 reviews the syntax and semantics of the pure rho calculus. Section 3 presents the cost-accounted extension. Section 4 gives the compositional translation. Section 5 verifies each of the five rewrite rules. Section 6 discusses implications for the Rholang architecture.

2 The Pure Rho Calculus

We recall the syntax and operational semantics of the rho calculus, presented here as a graph-structured lambda theory (GSLT).

2.1 Syntax

Processes and names are mutually defined:

$$P, Q \ ::= \ \mathbf{0} \ | \ \mathbf{for}(y \leftarrow x)\, P \ | \ x!\,(Q) \ | \ P \mid Q \ | \ *x \tag{1}$$

$$x, y \ ::= \ @P \tag{2}$$

The intended readings are:

0 The *stopped process*: does nothing.

for$(y \leftarrow x)\, P$
The process that *waits* for a message on channel x, binds the received name to y in P, and continues as the resulting instance of P. (In Rholang: `for( y <- x ) { P }`.)

$x!\,(Q)$
Sends the code of process Q on channel x. (In Rholang: `x!( Q )`.)

$P \mid Q$
The *parallel composition* of P and Q.

$*x$ *Dequotes* (reflects) channel x back into a running process.

$@P$
The *quote* (code) of process P, used as a channel name.

2.2 Free Names

The **for**-comprehension binds y. Free names are computed recursively:

$$\begin{aligned}
\mathrm{FN}(\mathbf{0}) &= \emptyset \\
\mathrm{FN}(\mathbf{for}(y \leftarrow x)\, P) &= \big(\mathrm{FN}(P) \setminus \{y\}\big) \cup \{x\} \\
\mathrm{FN}(x!\,(Q)) &= \mathrm{FN}(Q) \cup \{x\} \\
\mathrm{FN}(P \mid Q) &= \mathrm{FN}(P) \cup \mathrm{FN}(Q) \\
\mathrm{FN}(*x) &= \{x\}
\end{aligned}$$

2.3 Structural Equivalence

Structural equivalence, written $\equiv_S$, is the smallest equivalence relation on processes that includes α-equivalence and makes $(P, |, \mathbf{0})$ a commutative monoid:

$$P \mid \mathbf{0} \equiv_S P \qquad \text{(identity)}$$

$$P \mid Q \equiv_S Q \mid P \qquad \text{(commutativity)}$$

$$(P \mid Q) \mid R \equiv_S P \mid (Q \mid R) \qquad \text{(associativity)}$$

2.4 Rewrite Rules (Operational Semantics)

The single computational step is the Comm rule (communication):

$$\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q) \ \rightsquigarrow \ P\{@Q/y\} \tag{3}$$

subject to the contextual closure rules:

$$\frac{P \rightsquigarrow P'}{P \mid Q \rightsquigarrow P' \mid Q} \quad \text{(Par)} \tag{4}$$

$$\frac{P \equiv_S P' \qquad P' \rightsquigarrow Q' \qquad Q' \equiv_S Q}{P \rightsquigarrow Q} \quad \text{(Struct)} \tag{5}$$

Remark 2.1 (Semantic substitution)**.** *The notation $P\{u/v\}$ denotes* semantic *substitution. It coincides with ordinary syntactic substitution in all cases except dequotation, where:*

$$(*x)\{@Q/y\} \;=\; \begin{cases} Q & \text{if } y \equiv_N x, \\ *x & \text{otherwise,} \end{cases}$$

and $\equiv_N$ denotes name equality. See [1] for full details.

3 The Cost-Accounted Extension

We now present an extension of the rho calculus with digital signatures and a token-based cost-accounting mechanism. This extension captures the essential structure of phlogiston accounting in Rholang: processes carry signatures, tokens carry balances, and communication is gated by token consumption.

3.1 Extended Syntax

Definition 3.1 (Token and system syntax)**.**

$$T \;::=\; () \;\mid\; \mathfrak{s} \;\mid\; T{:}T \tag{6}$$

$$S \;::=\; P^{\mathfrak{s}} \;\mid\; T \;\mid\; S \mid S \tag{7}$$

$$\mathfrak{s} \;::=\; () \;\mid\; \mathrm{hash}(\langle \textit{signature} \rangle) \;\mid\; \mathfrak{s} \,\&\, \mathfrak{s} \tag{8}$$

where $P^{\mathfrak{s}}$ denotes a process P signed by a digital signature $\mathfrak{s}$.

The token $\mathfrak{s}{:}T$ is read as "the signature $\mathfrak{s}$ holds the remaining balance T."

3.2 Cost-Accounted Rewrite Rules

The five rewrite rules of the cost-accounted calculus are:

Rule 1 (single signature, whole redex):

$$(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}} \mid \mathfrak{s}{:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}} \mid T \tag{9}$$

Rule 2 (compound signature, whole redex, split tokens):

$$(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid \mathfrak{s}_1{:}T_1 \mid \mathfrak{s}_2{:}T_2 \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2 \tag{10}$$

Rule 3 (compound signature, whole redex, combined token):

$$(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid (\mathfrak{s}_1 \,\&\, \mathfrak{s}_2){:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T \tag{11}$$

Rule 4 (compound signature, split processes, combined token):

$$(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}_1} \mid (x!\,(Q))^{\mathfrak{s}_2} \mid (\mathfrak{s}_1 \,\&\, \mathfrak{s}_2){:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T \tag{12}$$

Rule 5 (compound signature, split processes, split tokens):

$$(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}_1} \mid (x!\,(Q))^{\mathfrak{s}_2} \mid \mathfrak{s}_1{:}T_1 \mid \mathfrak{s}_2{:}T_2 \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2 \tag{13}$$

Remark 3.2. *Rules 1–5 can be understood as variations on a single theme: a communication (the* Comm *rule of the pure calculus) that is* gated *by the consumption of a token whose signature matches the signature(s) of the communicating processes. The five rules differ in whether the redex is signed as a whole or its components are signed separately, and whether the token is combined or split across signatures.*

4 The Translation

The central observation is that the cost-accounting mechanism is a *fuel-acquisition protocol*: before a signed process can communicate, it must consume a token (fuel) from the channel associated with its signature. This protocol is expressible entirely within the pure rho calculus.

4.1 Signatures to Names

We define $\mathcal{N} : \mathfrak{s} \to \text{Names}$:

$$\mathcal{N}[\![()]\!] = @\mathbf{0} \tag{14}$$

$$\mathcal{N}[\![\text{hash}(\sigma)]\!] = @H_\sigma \quad \text{for a canonical process } H_\sigma \text{ encoding } \sigma \tag{15}$$

$$\mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!] = @\big(*\mathcal{N}[\![\mathfrak{s}_1]\!] \mid *\mathcal{N}[\![\mathfrak{s}_2]\!]\big) \tag{16}$$

The compound-signature case (16) exploits reflection: the name for $\mathfrak{s}_1 \,\&\, \mathfrak{s}_2$ is the quote of the parallel composition of the dequotations of the two component names. This ensures injectivity of the encoding.

4.2 Tokens to Processes

We define $\mathcal{T} : T \to \text{Processes}$:

$$\mathcal{T}[\![()]\!] = \mathbf{0} \tag{17}$$

$$\mathcal{T}[\![\mathfrak{s} : T]\!] = \mathcal{N}[\![\mathfrak{s}]\!]!\,(\mathcal{T}[\![T]\!]) \tag{18}$$

A token $\mathfrak{s}{:}T$ becomes a *message* on the channel $\mathcal{N}[\![\mathfrak{s}]\!]$ carrying the code of the remaining token T as its payload.

4.3 Signed Processes

The key idea: a signed process must *first consume fuel* from its signature channel before it can perform its intended communication. The leftover token (the payload of the fuel message) is released into the continuation via dequotation.

4.3.1 Atomic Signatures

$$\mathcal{P}[\![(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}}]\!] = \mathbf{for}(t \leftarrow \mathcal{N}[\![\mathfrak{s}]\!])\,\big(\mathbf{for}(y \leftarrow x)\,(*t \mid \mathcal{P}[\![P]\!])\big) \tag{19}$$

$$\mathcal{P}[\![(x!\,(Q))^{\mathfrak{s}}]\!] = \mathbf{for}(t \leftarrow \mathcal{N}[\![\mathfrak{s}]\!])\,\big(x!\,(\mathcal{P}[\![Q]\!]) \mid *t\big) \tag{20}$$

$$\mathcal{P}[\![(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}}]\!] = \mathbf{for}(t \leftarrow \mathcal{N}[\![\mathfrak{s}]\!])\,\big(\mathbf{for}(y \leftarrow x)\,(*t \mid \mathcal{P}[\![P]\!]) \mid x!\,(\mathcal{P}[\![Q]\!])\big) \tag{21}$$

In each case, the outermost **for** on $\mathcal{N}[\![\mathfrak{s}]\!]$ acts as a *fuel gate*: the process blocks until a token is available.

4.3.2 Compound Signatures

For $\mathfrak{s}_1 \,\&\, \mathfrak{s}_2$, the process must acquire fuel from *both* component channels:

$$\mathcal{P}[\![(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}_1 \& \mathfrak{s}_2}]\!] = \mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\,\mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\,\mathbf{for}(y \leftarrow x)\,(*t_1 \mid *t_2 \mid \mathcal{P}[\![P]\!]) \tag{22}$$

$$\mathcal{P}[\![(x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2}]\!] = \mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\,\mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\,\big(x!\,(\mathcal{P}[\![Q]\!]) \mid *t_1 \mid *t_2\big) \tag{23}$$

$$\mathcal{P}[\![(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2}]\!] = \mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\,\mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\,\big(\mathbf{for}(y \leftarrow x)\,(*t_1 \mid *t_2 \mid \mathcal{P}[\![P]\!]) \mid x!\,(\mathcal{P}[\![Q]\!])\big) \tag{24}$$

4.4 System Translation

The system translation $\mathcal{S}$ is defined compositionally:

$$\mathcal{S}[\![P^{\mathfrak{s}}]\!] = \mathcal{P}[\![P^{\mathfrak{s}}]\!] \tag{25}$$

$$\mathcal{S}[\![T]\!] = \mathcal{T}[\![T]\!] \tag{26}$$

$$\mathcal{S}[\![S_1 \mid S_2]\!] = \mathcal{S}[\![S_1]\!] \mid \mathcal{S}[\![S_2]\!] \tag{27}$$

4.5 Infrastructure Processes

Rules 2–5 involve situations where the granularity of the token (combined vs. split) does not match the granularity expected by the signed process. We introduce two families of *infrastructure processes* to mediate:

Definition 4.1 (Splitter). *The splitter converts a combined token for $\mathfrak{s}_1 \& \mathfrak{s}_2$ into separate tokens for $\mathfrak{s}_1$ and $\mathfrak{s}_2$:*

$$\mathrm{Split}(\mathfrak{s}_1, \mathfrak{s}_2) \;=\; \mathbf{for}(t \leftarrow \mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!])\,\big(\mathcal{N}[\![\mathfrak{s}_1]\!]!\,(\mathbf{0}) \mid \mathcal{N}[\![\mathfrak{s}_2]\!]!\,(*t)\big)$$

(In the persistent variant, the outermost **for** *is replicated.)*

Definition 4.2 (Joiner). *The joiner converts separate tokens for $\mathfrak{s}_1$ and $\mathfrak{s}_2$ into a combined token for $\mathfrak{s}_1 \,\&\, \mathfrak{s}_2$:*

$$\mathrm{Join}(\mathfrak{s}_1, \mathfrak{s}_2) \;=\; \mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\,\mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\,\mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!]!\,((*t_1 \mid *t_2))$$

Remark 4.3 (Persistent infrastructure). *In practice, splitters and joiners should be* persistent *(replicated) processes. The standard rho-calculus encoding of replication via self-referencing through reflection (cf. [1]) applies directly.*

5 Verification

We verify that each of the five cost-accounted rewrite rules is faithfully simulated by the translation.

5.1 Rule 1: Single Signature, Whole Redex

Source: $(\mathbf{for}(y \leftarrow x)\,P \mid x!\,(Q))^{\mathfrak{s}} \mid \mathfrak{s}{:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}} \mid T$

Translation:

$$\underbrace{\mathbf{for}(t \leftarrow \mathcal{N}[\![\mathfrak{s}]\!])\,\big(\mathbf{for}(y \leftarrow x)\,(*t \mid \mathcal{P}[\![P]\!]) \mid x!\,(\mathcal{P}[\![Q]\!])\big)}_{\mathcal{P}[\![(\mathbf{for}(y\leftarrow x)P \mid x!(Q))^{\mathfrak{s}}]\!]} \;\mid\; \underbrace{\mathcal{N}[\![\mathfrak{s}]\!]!\,(\mathcal{T}[\![T]\!])}_{\mathcal{T}[\![\mathfrak{s}:T]\!]}$$

Step 1 — fuel consumption on $\mathcal{N}[\![\mathfrak{s}]\!]$, binding $t \mapsto @\mathcal{T}[\![T]\!]$:

$$\mathbf{for}(y \leftarrow x)\,\big(*(@\mathcal{T}[\![T]\!]) \mid \mathcal{P}[\![P]\!]\big) \mid x!\,(\mathcal{P}[\![Q]\!]) \;\equiv_S\; \mathbf{for}(y \leftarrow x)\,\big(\mathcal{T}[\![T]\!] \mid \mathcal{P}[\![P]\!]\big) \mid x!\,(\mathcal{P}[\![Q]\!])$$

Step 2 — communication on x, binding $y \mapsto @\mathcal{P}[\![Q]\!]$:

$$\mathcal{T}[\![T]\!] \mid \mathcal{P}[\![P\{@Q/y\}]\!] \;=\; \mathcal{S}[\![(P\{@Q/y\})^{\mathfrak{s}} \mid T]\!] \quad \checkmark$$

5.2 Rule 2: Compound Signature, Whole Redex, Split Tokens

Source: $(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid \mathfrak{s}_1{:}T_1 \mid \mathfrak{s}_2{:}T_2 \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2$

Translation (using (24)):

$$\mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\, \mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\, \big(\mathbf{for}(y \leftarrow x)\,(*t_1 \mid *t_2 \mid \mathcal{P}[\![P]\!]) \mid x!\,(\mathcal{P}[\![Q]\!])\big) \mid \mathcal{N}[\![\mathfrak{s}_1]\!]!\,(\mathcal{T}[\![T_1]\!]) \;\mid\; \mathcal{N}[\![\mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T_2]\!])$$

Step 1 — fuel from $\mathcal{N}[\![\mathfrak{s}_1]\!]$: $t_1 \mapsto @\mathcal{T}[\![T_1]\!]$.
Step 2 — fuel from $\mathcal{N}[\![\mathfrak{s}_2]\!]$: $t_2 \mapsto @\mathcal{T}[\![T_2]\!]$.
Step 3 — Comm on x:

$$\mathcal{T}[\![T_1]\!] \mid \mathcal{T}[\![T_2]\!] \mid \mathcal{P}[\![P\{@Q/y\}]\!] \;=\; \mathcal{S}[\![(P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2]\!] \quad \checkmark$$

5.3 Rule 3: Compound Signature, Whole Redex, Combined Token

Source: $(\mathbf{for}(y \leftarrow x)\, P \mid x!\,(Q))^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid (\mathfrak{s}_1 \,\&\, \mathfrak{s}_2){:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T$

The whole-redex encoding (24) listens on both $\mathcal{N}[\![\mathfrak{s}_1]\!]$ and $\mathcal{N}[\![\mathfrak{s}_2]\!]$, but the combined token lives on $\mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!]$. The $\mathrm{Split}(\mathfrak{s}_1, \mathfrak{s}_2)$ infrastructure process (Definition 4.1) mediates:

$$\begin{aligned} &\mathrm{Split}(\mathfrak{s}_1, \mathfrak{s}_2) \mid \mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T]\!]) \\ \rightsquigarrow\quad &\mathcal{N}[\![\mathfrak{s}_1]\!]!\,(\mathbf{0}) \mid \mathcal{N}[\![\mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T]\!]) \end{aligned}$$

Now both fuel channels are populated and the reduction proceeds as in Rule 2, yielding $\mathbf{0} \mid \mathcal{T}[\![T]\!] \mid \mathcal{P}[\![P\{@Q/y\}]\!] \equiv_S \mathcal{S}[\![(P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T]\!]$. $\checkmark$

5.4 Rule 4: Split Processes, Combined Token

Source: $(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}_1} \mid (x!\,(Q))^{\mathfrak{s}_2} \mid (\mathfrak{s}_1 \,\&\, \mathfrak{s}_2){:}T \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T$

The translations of the separately signed processes listen on $\mathcal{N}[\![\mathfrak{s}_1]\!]$ and $\mathcal{N}[\![\mathfrak{s}_2]\!]$ respectively (equations (19) and (20)). The token lives on $\mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!]$. Again, the splitter mediates:

$$\mathrm{Split}(\mathfrak{s}_1, \mathfrak{s}_2) \mid \mathcal{N}[\![\mathfrak{s}_1 \,\&\, \mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T]\!]) \;\rightsquigarrow\; \mathcal{N}[\![\mathfrak{s}_1]\!]!\,(\mathbf{0}) \mid \mathcal{N}[\![\mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T]\!])$$

After fuel consumption, the remainder is $\mathbf{0} \mid \mathcal{T}[\![T]\!] \mid \mathcal{P}[\![P\{@Q/y\}]\!] \equiv_S \mathcal{S}[\![(P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T]\!]$. $\checkmark$

5.5 Rule 5: Split Processes, Split Tokens

Source: $(\mathbf{for}(y \leftarrow x)\, P)^{\mathfrak{s}_1} \mid (x!\,(Q))^{\mathfrak{s}_2} \mid \mathfrak{s}_1{:}T_1 \mid \mathfrak{s}_2{:}T_2 \;\longrightarrow\; (P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2$

Translation:

$$\mathbf{for}(t_1 \leftarrow \mathcal{N}[\![\mathfrak{s}_1]\!])\, \mathbf{for}(y \leftarrow x)\,(*t_1 \mid \mathcal{P}[\![P]\!]) \;\mid\; \mathbf{for}(t_2 \leftarrow \mathcal{N}[\![\mathfrak{s}_2]\!])\, \big(x!\,(\mathcal{P}[\![Q]\!]) \mid *t_2\big) \mid \mathcal{N}[\![\mathfrak{s}_1]\!]!\,(\mathcal{T}[\![T_1]\!]) \;\mid\; \mathcal{N}[\![\mathfrak{s}_2]\!]!\,(\mathcal{T}[\![T_2]\!])$$

Step 1 — both fuel channels fire:

$$\mathbf{for}(y \leftarrow x)\,(\mathcal{T}[\![T_1]\!] \mid \mathcal{P}[\![P]\!]) \;\mid\; x!\,(\mathcal{P}[\![Q]\!]) \mid \mathcal{T}[\![T_2]\!]$$

Step 2 — Comm on x:

$$\mathcal{T}[\![T_1]\!] \mid \mathcal{P}[\![P\{@Q/y\}]\!] \mid \mathcal{T}[\![T_2]\!] \;=\; \mathcal{S}[\![(P\{@Q/y\})^{\mathfrak{s}_1 \& \mathfrak{s}_2} \mid T_1 \mid T_2]\!] \quad \checkmark$$

6 Implications for Rholang Architecture

Because the rho calculus is the formal core of Rholang, the translation developed above is not merely a theoretical exercise — it is a *blueprint* for rearchitecting the phlogiston cost-accounting layer in Rholang 1.2 and beyond.

6.1 Cost Control as a Rholang Program

In the current architecture, phlogiston accounting is implemented as a privileged runtime extension: the Rholang virtual machine intercepts communications, checks balances, and debits tokens before allowing reduction to proceed. The translation shows that this entire mechanism can be *internalized* as a Rholang program.

Concretely:

- Each deploy's *signature* becomes a Rholang *channel name*, generated by quoting a canonical process that encodes the signer's identity.
- Each *token* (phlo balance) becomes a *message* on the corresponding signature channel.
- Each *signed process* is wrapped in a fuel-gate: a **for**-comprehension on its signature channel that must receive a token before the process can proceed.

6.2 Composable Metering

Because the accounting layer is now written in Rholang, it becomes *composable*. Different metering strategies can be implemented as different Rholang contracts:

- **Flat-rate metering:** A single token per communication.
- **Proportional metering:** Tokens whose continuation-payload encodes the remaining balance, debited proportionally to resource consumption.
- **Delegated metering:** Splitters and joiners (Section 4.5) allow one entity to fund another's computation by transferring tokens across signature channels.
- **Market-based metering:** Token prices can float based on demand for particular signature channels, implementing a fee market entirely within Rholang.

6.3 Formal Verification

The compositionality of the translation means it can be verified using standard techniques:

- **Bisimulation:** The translation preserves barbed bisimulation up to the infrastructure processes.
- **Full abstraction:** The ongoing Lean 4 formalization of the rho calculus can be extended to cover the cost-accounted extension and its translation.
- **Security:** The fuel-gate pattern ensures that no process can communicate without first acquiring a valid token, which is a capability-security property expressible and provable within the calculus.

6.4 Implementation Path

For the F1R3FLY platform, we envision the following implementation path:

1. **Phase 1:** Implement the translation as a source-to-source transformation in the Rholang 1.2 compiler, replacing the current runtime cost-accounting hooks.
2. **Phase 2:** Deploy splitters, joiners, and persistent infrastructure processes as system-level Rholang contracts on each F1R3Node shard.
3. **Phase 3:** Expose the signature-channel and token APIs to user contracts, enabling composable and user-definable metering.
4. **Phase 4:** Extend the Lean 4 formalization to cover the cost-accounted extension and verify the translation.

7 Conclusion

We have shown that the cost-accounted extension of the rho calculus — capturing the essential structure of Rholang's phlogiston system — can be faithfully translated back into the pure rho calculus. The translation is compositional and rests on a single idea: *signatures become channels, tokens become messages, and signed processes must consume fuel before they can communicate.*

For the F1R3FLY platform, this result is both a theoretical contribution and a practical blueprint. By internalizing cost accounting within Rholang itself, we eliminate the need for a privileged runtime extension, enable composable and formally verifiable metering, and open the door to user-definable fee structures — all without leaving the expressive universe of the rho calculus.

E Pluribus Potentia

References

[1] L. G. Meredith and M. Radestock, "A reflective higher-order calculus," *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.

[2] R. Milner, *Communicating and Mobile Systems: the π-Calculus*, Cambridge University Press, 1999.

[3] L. G. Meredith *et al.*, "Rholang Specification," F1R3FLY.io / RChain Cooperative, 2017–2026.